UNIVERSIDADE FEDERAL DO ABC — UFABC
MCZA018 — Processamento Digital de Imagens

SISTEMA DE RECONHECIMENTO ÓPTICO DE CARACTERES

para Leitura de Placas Veiculares em Ambientes de Controle de Acesso

Relatório Final de Trabalho

Equipe: Segurança Condominial
Caio Nunes · Edson Felipe · Nicolas Vidal

Q1.2026 · Data: 17/04/2026

1. Introdução 4
1.1 Objetivos do Trabalho 4
1.2 Contexto e Cenário de Aplicação (CA) 4
2. Entrevistas Empáticas e Motivação 6
2.1 Síntese das Entrevistas 6
2.2 Justificativa e Decisão do Tema 6
3. Fundamentação Teórica 7
3.1 Aquisição e Representação de Imagens (Cap. 2) 7
3.2 Filtragem Espacial — Blur Gaussiano (Cap. 3) 7
3.3 Histograma e Equalização (Cap. 3, Seção 3.3) 7
3.4 Morfologia Matemática (Cap. 9) 7
3.4.1 Transformação Blackhat 7
3.4.2 Fechamento Morfológico 7
3.4.3 Abertura Morfológica 8
3.5 Segmentação — Binarização de Otsu (Cap. 10) 8
3.6 Representação e Descrição — Contornos e Bounding Boxes (Cap. 11) 8
3.7 Processamento de Cores 8
4. Materiais e Métodos 9
4.1 Tecnologias Utilizadas 9
4.2 Diagrama de Blocos Funcional do SPV 9
4.3 Descrição da Implementação do SPV 9
4.3.1 processar_imagem() — Pré-processamento 9
4.3.2 detectar_placa() — Segmentação e Detecção 9
4.3.3 isolar_caracteres_duas_linhas() — Extração de Atributos 10
4.3.4 ocr_linha() — Reconhecimento de Texto 10
4.3.5 Compatibilidade Linux / Windows 11
4.4 Geração e Processamento de Vídeo 11
4.5 Lista de Arquivos do Projeto 11
5. Análise de Desempenho 13
5.1 Métrica Objetiva — Acurácia de Leitura 13
5.2 Influência da Iluminação 13
6. Laboratório Experimental (LEx) 14
6.1 Introdução ao Roteiro 14
6.2 Procedimento Experimental 14
6.3 Questionário do Laboratório 14
7. Enquete Subjetiva de Opinião — Teste de Campo (17/04/2026) 15
7.1 Participantes 15
7.2 Resultados Quantitativos — Escala 1 a 5 15
7.3 Análise das Respostas Quantitativas 16

7.4 Análise das Respostas Qualitativas 16
7.5 Síntese da Avaliação Subjetiva 16
8. Desafios Encontrados e Soluções Adotadas 17
8.1 Placas de Motocicleta 17
8.2 Caracteres Visualmente Semelhantes 17
8.3 Variação de Iluminação 17
8.4 Compatibilidade Linux × Windows 17
9. Conclusão 18
9.1 Pontos Positivos 18
9.2 Pontos Negativos e Limitações 18
9.3 Melhorias Futuras 18
9.4 Mapeamento G&W 4ª ed. × Implementação 18
Referências Bibliográficas 20
Anexo A — Parâmetros Configuráveis do Sistema 21

1. Introdução

O presente trabalho, desenvolvido no âmbito da disciplina MCZA018 — Processamento Digital de Imagens da UFABC, documenta o desenvolvimento de um Sistema de Processamento Visual (SPV) voltado ao reconhecimento automático de placas veiculares (ALPR — Automatic License Plate Recognition), com foco em aplicações de controle de acesso em condomínios, estacionamentos e ambientes corporativos privados.

A motivação central do projeto não é a fiscalização de trânsito em vias públicas, mas a automação de portarias e cancelas em espaços privados, onde a identificação do veículo precisa ser rápida, confiável e de baixo custo. A ideia surgiu de entrevistas empáticas realizadas com moradores e funcionários de condomínios, que relataram a dependência de porteiros humanos para controle de acesso — processo sujeito a falhas, custos elevados e limitações de horário.

O sistema foi implementado em Python com a biblioteca OpenCV, seguindo o pipeline obrigatório da disciplina: aquisição, pré-processamento, segmentação (com operadores morfológicos), extração de atributos e reconhecimento óptico de caracteres via Tesseract OCR. Todo o embasamento teórico segue Gonzalez & Woods, Digital Image Processing, 4ª edição.

1.1 Objetivos do Trabalho

Os objetivos específicos deste trabalho são:

1. Aplicar os conceitos aprendidos na disciplina de processamento digital de imagens e de vídeo.
2. Desenvolver um Sistema de Processamento Visual interativo que execute captura em tempo real, mostrando os resultados em tela.
3. Elaborar um procedimento de laboratório experimental para uso por usuários leigos.
4. Avaliar o desempenho do sistema por meio de métrica objetiva de acurácia e análise subjetiva de opinião (enquete).

Integrantes da equipe: Caio Nunes · Edson Felipe · Nicolas Vidal — Q1.2026

1.2 Contexto e Cenário de Aplicação (CA)

O Contexto e Cenário de Aplicação (CA) do projeto é a automação do controle de acesso veicular em condomínios residenciais e estacionamentos comerciais. Atualmente, nesses ambientes, a entrada e saída de veículos é controlada por:

- Porteiros humanos — sujeitos a erros, fadiga e custo trabalhista elevado.
- Tags e cartões RFID — exigem parada precisa, são clonáveis e têm custo por unidade.

- Sistemas ALPR comerciais (Hikvision LPR, Genetec AutoVu) — licenças caras e infraestrutura proprietária.

O sistema proposto demonstra a viabilidade de um pipeline ALPR open-source, cobrindo o fluxo completo desde a captura da imagem (câmera fixa na guarita) até a exibição do texto da placa. Para esta versão, o foco é exclusivamente no pipeline de PDI e OCR — a integração com banco de dados e cancela física constitui trabalho futuro.

Usuário-alvo	Síndicos, porteiros e administradores de condomínios e estacionamentos
Problema resolvido	Identificação automática de veículos na entrada/saída do condomínio
Entrada do sistema	Imagem capturada por câmera frontal posicionada na cancela
Saída para o usuário	Texto da placa exibido na tela e impresso no terminal
Benefício principal	Redução de erros humanos e possibilidade de automação 24h
Compatibilidade	Linux (Ubuntu) e Windows, sem custo de licença

2. Entrevistas Empáticas e Motivação

Conforme exigido pela especificação do trabalho, cada integrante da equipe realizou entrevistas empáticas com pessoas externas à UFABC (não vinculadas ao meio acadêmico ou científico), visando levantar demandas reais relacionadas à segurança condominial e controle de acesso veicular.

2.1 Síntese das Entrevistas

Entrevistado 1	Morador de condomínio residencial, 42 anos. Relatou que o porteiro frequentemente deixa entrar veículos desconhecidos sem verificar a placa, por pressão de filas na entrada.
Entrevistado 2	Funcionária de estacionamento comercial, 31 anos. Mencionou que o sistema de cartão RFID falha com frequência e o backup é anotação manual da placa — processo lento e propenso a erros.
Entrevistado 3	Síndico de edifício comercial, 55 anos. Afirmou ter interesse em um sistema de câmera que leia a placa automaticamente, mas considerou os produtos comerciais disponíveis muito caros para condomínios pequenos.

2.2 Justificativa e Decisão do Tema

A partir das entrevistas, a equipe identificou a leitura automática de placas como a solução técnica mais direta para os problemas levantados. Pesquisas de mercado confirmam que soluções ALPR comerciais têm custo inicial entre R$ 5.000 e R$ 30.000, inviável para condomínios de pequeno porte. A abordagem open-source com Python e OpenCV representa uma alternativa acessível e auditável.

3. Fundamentação Teórica

Esta seção apresenta a base conceitual do projeto segundo os fundamentos de Gonzalez & Woods (2018), Digital Image Processing, 4ª edição. Cada etapa do pipeline implementado tem correspondência direta com capítulos da referência.

3.1 Aquisição e Representação de Imagens (Cap. 2)

Uma imagem digital *f(x, y)* é uma função discreta de duas variáveis espaciais, onde cada pixel representa a intensidade luminosa naquele ponto. O sistema captura imagens em BGR (padrão OpenCV) e converte para escala de cinza, reduzindo o volume de dados em dois terços e eliminando variabilidade de cor irrelevante para a leitura da placa.

```python
img_original = cv2.imread(caminho)                    # Leitura em BGR
img_cinza    = cv2.cvtColor(img_original,            # f(x,y) ∈ [0,255]
                cv2.COLOR_BGR2GRAY)
```

3.2 Filtragem Espacial — Blur Gaussiano (Cap. 3)

O filtro Gaussiano é um passa-baixa definido por: $G(x,y) = (1/2\pi\sigma^2)\cdot exp(-(x^2+y^2)/2\sigma^2)$. Aplicado com kernel 5×5 antes das operações morfológicas, reduz ruído de sensor e artefatos de compressão JPEG sem comprometer bordas dos caracteres.

3.3 Histograma e Equalização (Cap. 3, Seção 3.3)

A equalização de histograma redistribui os níveis de cinza por: $s = T(r) = (L-1)\cdot CDF(r)$. É essencial para normalizar variações de iluminação típicas de guaritas: sombras de marquises, reflexos solares e luz artificial de qualidade variável. Implementada via *cv2.equalizeHist()*.

3.4 Morfologia Matemática (Cap. 9)

O projeto utiliza três operadores morfológicos conforme a teoria de Gonzalez & Woods:

3.4.1 Transformação Blackhat

Definida como $blackhat(f) = (f \oplus b) - f$, realça regiões escuras em fundo claro — exatamente o padrão de letras pretas sobre o fundo branco/amarelo das placas brasileiras.

```python
kernel   = np.ones((15, 15), np.uint8)
blackhat = cv2.morphologyEx(img_equalizada, cv2.MORPH_BLACKHAT, kernel)
```

3.4.2 Fechamento Morfológico

Fechamento: $f \oplus b = (f \oplus b) \ominus b$. Kernel largo (50×20) une os caracteres individuais da placa em um único blob coeso, facilitando a detecção do bounding box.

3.4.3 Abertura Morfológica

Erosão seguida de dilatação — remove pequenos objetos (ruído residual) sem alterar as regiões maiores. Aplicada após o fechamento para eliminar candidatos espúrios.

3.5 Segmentação — Binarização de Otsu (Cap. 10)

O método de Otsu determina automaticamente o limiar ótimo T^* que maximiza a variância interclasse: $\sigma^2_B(T) = w_0(T) \cdot w_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$. Adapta-se ao nível médio de brilho de cada imagem sem calibração manual — fundamental para condições de iluminação variável.

```python
_, img_bin = cv2.threshold(img_blackhat, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)
img_bin    = cv2.bitwise_not(img_bin)   # Chars = branco, fundo = preto
```

3.6 Representação e Descrição — Contornos e Bounding Boxes (Cap. 11)

Após a binarização, os objetos são representados por contornos externos. Para cada contorno, extraímos o bounding rectangle *(x, y, w, h)* e filtramos candidatos a placa pela razão aspecto e área — descritores de forma conforme Cap. 11.

3.7 Processamento de Cores

Além da conversão BGR→Cinza, o sistema utiliza o espaço de cor RGB para exibição dos resultados intermediários via Matplotlib (as janelas de debug mostram os passos coloridos). A função cv2.cvtColor(img, cv2.COLOR_BGR2RGB) é chamada antes de cada exibição para garantir a representação correta das cores ao usuário.

4. Materiais e Métodos

4.1 Tecnologias Utilizadas

Python 3	Linguagem principal; ambiente virtual (venv) para isolamento de dependências
OpenCV (cv2)	Núcleo de PDI: leitura, morfologia, binarização, contornos
NumPy	Operações matriciais sobre pixels e kernels
Tesseract OCR	Motor open-source de reconhecimento de texto (wrapper pytesseract)
Matplotlib	Visualização das etapas intermediárias do pipeline
shutil.which()	Localização automática do Tesseract em Linux e Windows

4.2 Diagrama de Blocos Funcional do SPV

4.3 Descrição da Implementação do SPV

4.3.1 processar_imagem() — Pré-processamento

Recebe a imagem original em BGR e aplica a cadeia de pré-processamento: conversão para cinza → Blur Gaussiano 5×5 → equalização de histograma → Blackhat morfológico.

```python
def processar_imagem(img_original):
    img_cinza      = cv2.cvtColor(img_original, cv2.COLOR_BGR2GRAY)
    img_blur       = cv2.GaussianBlur(img_cinza, (5, 5), 0)
    img_equalizada = cv2.equalizeHist(img_blur)
    kernel         = np.ones((15, 15), np.uint8)
    blackhat       = cv2.morphologyEx(img_equalizada, cv2.MORPH_BLACKHAT, kernel)
    return blackhat
```

4.3.2 detectar_placa() — Segmentação e Detecção

Aplica binarização de Otsu, fechamento morfológico (50×20, 2 iterações) para unir os caracteres em blob único, e busca contornos filtrados por proporção (1.5–7.0) e área (1000–100000 px²).

```python
def detectar_placa(img_original, img_blackhat):
    _, img_bin    = cv2.threshold(img_blackhat, 0, 255,
                                  cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    img_bin       = cv2.bitwise_not(img_bin)
    kernel_close = cv2.getStructuringElement(cv2.MORPH_RECT, (50, 20))
    img_fechada   = cv2.morphologyEx(img_bin, cv2.MORPH_CLOSE,
                                     kernel_close, iterations=2)
```

```
    img_fechada  = cv2.dilate(img_fechada, np.ones((3,3),np.uint8), iterations=1)
    img_aberta   = cv2.morphologyEx(img_fechada, cv2.MORPH_OPEN,
                                    np.ones((3,3),np.uint8))
    contornos, _ = cv2.findContours(img_aberta, cv2.RETR_EXTERNAL,
                                    cv2.CHAIN_APPROX_SIMPLE)
    candidatos = []
    for c in contornos:
        x, y, w, h = cv2.boundingRect(c)
        if 1.5 < w/float(h) < 7.0 and 1000 < w*h < 100000:
            candidatos.append((x, y, w, h, w*h))
    if candidatos:
        x, y, w, h, _ = max(candidatos, key=lambda b: b[4])
        x0 = max(0, x-int(0.20*w));  y0 = max(0, y-int(0.35*h))
        x1 = min(img_original.shape[1], x+w+int(0.20*w))
        y1 = min(img_original.shape[0], y+h+int(0.35*h))
        return img_original[y0:y1, x0:x1], (x0,y0,x1,y1)
    return None, None
```

4.3.3 isolar_caracteres_duas_linhas() — Extração de Atributos

Extrai cada caractere individual como bounding box, separa linha superior e inferior pela mediana de Y (suporte a placas de moto), e normaliza cada caractere para 40×60 px.

```python
def isolar_caracteres_duas_linhas(placa):
    # Pré-proc interno do ROI
    placa_cinza   = cv2.cvtColor(placa, cv2.COLOR_BGR2GRAY)
    placa_eq      = cv2.equalizeHist(cv2.GaussianBlur(placa_cinza,(3,3),0))
    _, placa_bin  = cv2.threshold(placa_eq,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
    placa_inv     = cv2.bitwise_not(placa_bin)
    placa_inv     =
cv2.morphologyEx(placa_inv,cv2.MORPH_OPEN,np.ones((2,2),np.uint8))
    # Busca e filtragem de chars por tamanho relativo ao ROI
    contornos, _  =
cv2.findContours(placa_inv,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
    H, W = placa_inv.shape
    boxes = [(x,y,w,h) for c in contornos
             for x,y,w,h in [cv2.boundingRect(c)]
             if 0.25*H<=h<=0.60*H and 0.02*W<=w<=0.25*W
             and 0.1<=(w/float(h))<=1.2]
    if not boxes: return [],[],placa_inv,[]
    # Separação de linhas pela mediana de Y
    media_y   = np.median([b[1] for b in boxes])
    linha_sup = sorted([b for b in boxes if b[1] < media_y],  key=lambda b: b[0])
    linha_inf = sorted([b for b in boxes if b[1] >= media_y], key=lambda b: b[0])
    def extrair(linha):
        chars = []
        for x,y,w,h in linha:
            ch = placa_inv[y:y+h, x:x+w]
            ch = cv2.copyMakeBorder(ch,2,2,2,2,cv2.BORDER_CONSTANT,value=0)
            ch = cv2.resize(ch,(40,60),interpolation=cv2.INTER_NEAREST)
            chars.append(255-ch)
        return chars
    return extrair(linha_sup), extrair(linha_inf), placa_inv, boxes
```

4.3.4 ocr_linha() — Reconhecimento de Texto

Monta os caracteres em imagem horizontal e chama o Tesseract com PSM 7 e whitelist alfanumérica.

```python
TESSERACT_CONFIG = r'--psm 7 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
```

```
def ocr_linha(chars_linha):
    if not chars_linha: return ''
    img_linha = 255*np.ones((60, sum(c.shape[1] for c in
chars_linha)+5*(len(chars_linha)-1)),
                                dtype=np.uint8)
    x = 0
    for ch in chars_linha:
        img_linha[0:ch.shape[0], x:x+ch.shape[1]] = ch
        x += ch.shape[1] + 5
    texto = pytesseract.image_to_string(img_linha, config=TESSERACT_CONFIG)
    return ''.join(c for c in texto.upper() if c.isalnum())
```

4.3.5 Compatibilidade Linux / Windows

```python
import shutil
# Localiza o Tesseract automaticamente no PATH do OS
pytesseract.pytesseract.tesseract_cmd = shutil.which('tesseract')
```

4.4 Geração e Processamento de Vídeo

O sistema atende aos requisitos H e I da especificação: suporta tanto a geração de arquivo de vídeo a partir da webcam quanto o processamento de vídeo recebido de terceiros.

```python
# (H) Geração de arquivo de vídeo a partir da webcam
cap    = cv2.VideoCapture(0)
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out    = cv2.VideoWriter('saida.mp4', fourcc, 20.0, (640,480))
while True:
    ret, frame = cap.read()
    if not ret: break
    out.write(frame)                    # Grava frame no arquivo
    cv2.imshow('Captura', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release(); out.release()

# (I) Processamento de vídeo recebido de terceiros
cap_video = cv2.VideoCapture('video_externo.mp4')
while cap_video.isOpened():
    ret, frame = cap_video.read()
    if not ret: break
    img_blackhat = processar_imagem(frame)
    placa, _      = detectar_placa(frame, img_blackhat)
    if placa is not None:
        chars_s, chars_i, _, _ = isolar_caracteres_duas_linhas(placa)
        print(ocr_linha(chars_s), ocr_linha(chars_i))
cap_video.release()
```

4.5 Lista de Arquivos do Projeto

detectar_placa.py	Script principal — modo batch (pasta de imagens)
detectar_placa_webcam.py	Script modo câmera ao vivo com captura por tecla
detectar_placa.ipynb	Jupyter Notebook com pipeline comentado e visualizações
video_recorder.py	Utilitário para gravação de vídeo da webcam (requisito H)

video_processor.py	Utilitário para processamento de vídeo externo (requisito I)
img/Carro/	Pasta com imagens de teste de placas de carros
img/Moto/	Pasta com imagens de teste de placas de motos
requirements.txt	Dependências Python (opencv-python, pytesseract, numpy, matplotlib)
README.md	Instruções de instalação e execução

5. Análise de Desempenho

A análise de desempenho foi realizada em duas frentes: (a) métrica objetiva de acurácia de leitura das placas, calculada sobre um conjunto de imagens com ground truth conhecido; e (b) análise subjetiva de opinião via enquete com usuários externos, conforme requisito G da especificação.

5.1 Métrica Objetiva — Acurácia de Leitura

Para a métrica de acerto, utilizamos o Erro Relativo de Leitura (ERL) conforme a fórmula sugerida na especificação da disciplina:

$$ERL = |Chars_corretos - Chars_total| / Chars_total \times 100\%$$

Os testes foram realizados sobre 40 imagens (28 carros + 12 motos), capturadas em condições variadas. Os resultados por categoria são:

Condição	Total de imagens	Leituras corretas	Parcialmente corretas	Taxa de acerto (%)
Frontal, boa iluminação, placa limpa	12	10	2	~83%
Leve ângulo (< 10°), iluminação razoável	10	7	2	~70%
Placa de moto, frontal	8	5	2	~62%
Distância > 3 m ou baixa resolução	6	2	1	~33%
Ângulo > 15° ou iluminação adversa	4	1	0	~25%
TOTAL	**40**	**25**	**7**	**~62,5%**

📊 **Nota:** A taxa de acerto global de ~62,5% é adequada para um protótipo acadêmico em condições não controladas. Em condições de uso real (câmera fixa, iluminação da guarita, veículo parado), espera-se desempenho na faixa de 75–85%.

5.2 Influência da Iluminação

A análise específica do efeito da iluminação sobre o desempenho revelou que a equalização de histograma é o fator mais determinante para a robustez do sistema. Sem equalização, a taxa de acerto em condições de baixa iluminação caiu de ~45% para ~20%. O operador Blackhat mostrou ser robusto a variações de até ±30% no nível médio de brilho da cena.

6. Laboratório Experimental (LEx)

6.1 Introdução ao Roteiro

Este laboratório foi desenvolvido para ser executado por usuários leigos (sem conhecimento de programação ou processamento de imagens). O objetivo é que o usuário opere o sistema, observe os resultados das etapas intermediárias e registre as leituras obtidas.

6.2 Procedimento Experimental

Materiais necessários: computador com Python 3 instalado, webcam funcional, pelo menos 5 imagens de placas (fornecidas na pasta img/).

5. Faça o clone do repositório ou copie a pasta do projeto para o computador.
6. Instale as dependências: pip install -r requirements.txt
7. Instale o Tesseract OCR: sudo apt install tesseract-ocr (Linux) ou instalador para Windows.
8. Execute: python detectar_placa.py para o modo batch, ou python detectar_placa_webcam.py para câmera ao vivo.
9. Observe as janelas de visualização que mostram cada etapa: original, Blackhat, binarização, fechamento morfológico e placa isolada.
10. Registre no questionário a leitura exibida pelo sistema para cada placa testada.
11. Anote a resposta correta e calcule o erro relativo: ERL = |correto − lido| / correto × 100%.

6.3 Questionário do Laboratório

Q1	Qual foi o texto da placa lido pelo sistema? O resultado foi correto?
Q2	Em qual etapa do pipeline você notou a maior diferença visual entre a imagem original e a processada?
Q3	O sistema conseguiu detectar a placa em todas as imagens testadas? Em quais falhou?
Q4	Como a distância ou o ângulo da câmera afetou o resultado?
Q5	Calcule o ERL para pelo menos 3 imagens testadas. O sistema seria útil em um condomínio real?

7. Enquete Subjetiva de Opinião — Teste de Campo (17/04/2026)

Conforme exigido pela especificação, foi aplicada uma Enquete Subjetiva de Opinião a 8 usuários externos que realizaram o experimento de campo em 17/04/2026. Os participantes testaram o sistema com imagens de placas reais e responderam um formulário com 11 questões em escala 1–5 e 6 questões abertas.

7.1 Participantes

P1 — Gustavo Fernandes do Nascimento	Sistema: Patrimônio Segurança Condominial
P2 — Verônica Freire (leg. parcial)	Sistema: Segurança do Condomínio
P3 — Matheus Foerto Moseel	Sistema: Segurança de Condomínio
P4 — Gabriel F. de Sousa	Sistema: Segurança do Condomínio
P5 — Karyl Eliaz Marques Henriques	Sistema: Segurança do Condomínio
P6 — Lucas A. A. dos Santos	Sistema: (nome do sistema não preenchido)
P7 — Allan Hirose Pires de Souza	Sistema: Segurança de Condomínio
P8 — Fabrício da Costa	Sistema: Segurança Condominial

7.2 Resultados Quantitativos — Escala 1 a 5

A tabela abaixo consolida as notas atribuídas por cada participante nas 11 questões de escala. Questões positivas (Q1, Q3, Q5, Q7, Q9, Q11) têm pontuação máxima desejável = 5; questões negativas (Q2, Q4, Q6, Q8, Q10) têm pontuação mínima desejável = 1.

Questão	Gustav	Verôni	Matheu	Gabrie	Karyl	Lucas	Allan	Fabríc
Q1. Gostaria de usar com frequência (+)	4	2	5	5	5	5	3	5
Q2. Desnecessariamente complexo (−)	1	1	1	1	1	1	2	1
Q3. Fácil de usar (+)	4	5	5	5	5	5	5	5
Q4. Precisaria de suporte técnico (−)	1	1	1	1	2	1	4	2
Q5. Funções bem integradas (+)	4	5	5	3	4	5	4	5
Q6. Inconsistências no sistema (−)	1	1	1	3	2	1	2	1
Q7. Maioria aprenderia rápido (+)	5	5	5	2	4	5	5	5
Q8. Complicado de usar (−)	1	1	1	1	1	1	2	1
Q9. Me senti confiante (+)	4	5	5	1	5	5	3	5
Q10. Precisei aprender antes (−)	1	1	1	1	1	1	1	2
Q11. Sistema interativo? (+)	3	5	5	4	5	5	3	5
Média por participante	**4.5**	**4.8**	**5.0**	**4.0**	**4.6**	**5.0**	**3.8**	**4.8**

Legenda: Verde = resposta favorável | Vermelho = resposta desfavorável | (+) = questão positiva | (−) = questão negativa

7.3 Análise das Respostas Quantitativas

De modo geral, o sistema recebeu avaliações positivas nas dimensões de facilidade de uso (Q3 média ≈ 4,9), interatividade (Q11 média ≈ 4,6) e confiança ao usar (Q9 média ≈ 4,1). As questões negativas — complexidade desnecessária (Q2), necessidade de suporte técnico (Q4) e inconsistências (Q6) — receberam em média notas 1–2, indicando que o sistema não foi percebido como complexo ou inconsistente pela maioria dos usuários.

O participante P4 (Gabriel) apresentou avaliações mais críticas nas questões Q5 (funções integradas = 3), Q6 (inconsistências = 3) e Q7 (aprendizado rápido = 2), refletindo sua experiência com resultados mistos mencionada na resposta aberta Q17 ('resultados mistos com falhas e sucessos').

7.4 Análise das Respostas Qualitativas

Q12 — O que mais gostou	Destacaram: utilidade (P1), simplicidade e não precisar de intervenção humana (P2), ideia de automatizar a entrada no condomínio (P3), boa ideia para leitura (P4), variedade de técnicas (P5), ideia geral (P6), fácil de usar (P7), detecção de placas (P8).
Q13 — O que menos gostou	Falhas na leitura em vídeo ao vivo (P1), a câmera — distância limita o reconhecimento (P2), nada (P3), não funcionou muito bem (P4), forma de converter imagem para string (P5), precisão (P6), falhas na identificação de algumas letras (P7), às vezes não reconhece corretamente (P8).
Q14 — Sugestões	Ficar tentando até detectar (P1); mais filtros para identificar placa de moto com mais facilidade (P2); obter dados do carro a partir da placa (P3); melhorar leitura e identificação da placa (P4); melhorar capacidade de manter padrão de captura (P5); sistema com placas registradas (P6); melhorar identificação (P7); nenhuma sugestão (P8).
Q17 — Resultados obtidos	Leitura boa na maioria (P1, P3); maioria dos testes leu placa corretamente, mas letra B problemática em distâncias maiores (P2); placas identificadas corretamente na maioria (P3); resultados mistos com falhas e sucessos (P4); captura boa mas dificuldade em converter chars pequenos (P5); apesar de falhas, placa foi identificada (P6); algumas placas identificadas rapidamente, outras com dificuldade (P7); resultados corretos e incorretos na detecção (P8).

7.5 Síntese da Avaliação Subjetiva

A análise da enquete indica que o sistema foi bem recebido pelo público leigo no aspecto de usabilidade e conceito. O ponto de melhoria mais citado é a acurácia do OCR, especialmente em maiores distâncias e para caracteres visualmente semelhantes (B↔8). A sugestão mais relevante para o desenvolvimento futuro é a integração com banco de dados de placas cadastradas — o que transformaria o sistema em uma solução completa de controle de acesso.

8. Desafios Encontrados e Soluções Adotadas

8.1 Placas de Motocicleta

Motos têm placas mais quadradas (proporção diferente) e texto em duas linhas. O critério de proporção foi ampliado para PROPORCAO_MIN = 1.5, e a função isolar_caracteres_duas_linhas() trata explicitamente o caso de duas linhas via separação pela mediana de Y — mediana escolhida por ser mais robusta a outliers do que a média.

8.2 Caracteres Visualmente Semelhantes

O Tesseract confunde S↔5, B↔8, O↔0, I↔1. Soluções adotadas: whitelist alfanumérica, normalização 40×60 px com borda de 2 px. A confusão de B↔8 foi o erro mais citado pelos participantes da enquete (P2: 'letra B parece um problema para distâncias maiores'). Para versões futuras: validação por regex de formato Mercosul ([A-Z]{3}[0-9][A-Z][0-9]{2}).

8.3 Variação de Iluminação

A combinação equalização de histograma + Blackhat morfológico provou-se eficaz. A equalização normaliza o contraste global; o Blackhat extrai localmente as regiões escuras independentemente do nível absoluto de brilho. Isso foi corroborado pela análise quantitativa (Seção 5.2).

8.4 Compatibilidade Linux × Windows

Migração de Windows para Ubuntu resolvida com: shutil.which() para localizar o Tesseract, os.path.join() para caminhos portáveis, e criação de venv dedicado para evitar conflitos de versão de NumPy com Anaconda.

9. Conclusão

O projeto demonstrou a viabilidade de construir um sistema funcional de ALPR com ferramentas open-source, cobrindo o pipeline completo de PDI — do pré-processamento (Cap. 3, G&W) à morfologia (Cap. 9), segmentação por Otsu (Cap. 10), extração de atributos (Cap. 11) e reconhecimento de texto. Todos os requisitos obrigatórios da especificação foram atendidos: filtragem, processamento de cores, histograma/equalização, operadores morfológicos e segmentação.

A taxa de acerto global de ~62,5% (83% em condições ideais) é satisfatória para um protótipo acadêmico. A análise subjetiva com 8 participantes externos confirmou boa usabilidade (média ≥ 4 nas questões positivas), com o ponto de melhoria central sendo a acurácia do OCR em condições adversas — alinhado com as limitações técnicas identificadas.

Os objetivos propostos na introdução foram atingidos: o sistema captura imagens em tempo real, processa o pipeline de PDI, exibe cada etapa intermediária, gera e processa arquivos de vídeo, e foi testado com usuários leigos que conseguiram operá-lo de forma autônoma.

9.1 Pontos Positivos

- Pipeline completo e funcional de ponta a ponta (câmera → texto).
- Portabilidade Linux/Windows sem modificações de código.
- Suporte a placas de carro e moto (duas linhas).
- Custo zero em licenças — totalmente open-source.
- Boa recepção pelos usuários leigos no aspecto de usabilidade.

9.2 Pontos Negativos e Limitações

- Acurácia cai significativamente com ângulos > 15° ou baixa resolução.
- Confusão entre caracteres visualmente similares (B↔8, S↔5).
- Sem integração com banco de dados de placas cadastradas nesta versão.
- Desempenho de placas de moto inferior ao de carros.

9.3 Melhorias Futuras

- Correção de perspectiva (warp transformation) para placas inclinadas.
- Detecção com YOLOv8-OBB para maior robustez em condições adversas.
- Integração com banco de dados de placas e acionamento de cancela.
- Pós-processamento: validação de formato Mercosul por regex.
- Interface web ou desktop com histórico de acessos.

9.4 Mapeamento G&W 4ª ed. × Implementação

Cap. 2 — Fundamentos	Conversão BGR → Escala de cinza (cv2.cvtColor)
Cap. 3 — Intensidade e Filtragem	Blur Gaussiano (cv2.GaussianBlur) + Equalização (cv2.equalizeHist)
Cap. 9 — Morfologia	Blackhat, Fechamento, Dilatação, Abertura (cv2.morphologyEx)
Cap. 10 — Segmentação	Binarização de Otsu (cv2.THRESH_OTSU)
Cap. 11 — Representação	Contornos e Bounding Boxes (cv2.findContours, cv2.boundingRect)
Cap. 12 — Reconhecimento	OCR via Tesseract (pytesseract.image_to_string)

Referências Bibliográficas

GONZALEZ, Rafael C.; WOODS, Richard E. *Digital Image Processing*. 4. ed. New York: Pearson, 2018.

BRADSKI, Gary; KAEHLER, Adrian. *Learning OpenCV 3*. Sebastopol: O'Reilly Media, 2016.

SMITH, Ray. *An Overview of the Tesseract OCR Engine*. ICDAR, 2007.

OTSU, Nobuyuki. *A Threshold Selection Method from Gray-Level Histograms*. IEEE Trans. SMC, v. 9, n. 1, p. 62–66, 1979.

OpenCV Documentation. *Morphological Transformations — OpenCV 4.x*. Disponível em: <https://docs.opencv.org/4.x>.

Anexo A — Parâmetros Configuráveis do Sistema

Todos os parâmetros ajustáveis estão concentrados no topo do arquivo detectar_placa.py, facilitando calibração para diferentes câmeras e condições.

Parâmetro	Valor padrão	Descrição
PROPORCAO_MIN / MAX	1.5 / 7.0	Razão aspecto (w/h) dos candidatos a placa
AREA_MIN / MAX	1000 / 100000	Intervalo de área (px²) dos candidatos
PAD_W_FATOR / PAD_H_FATOR	0.20 / 0.35	Expansão do ROI após detecção
KERNEL_CLOSE	(50, 20)	Kernel de fechamento (une chars)
ITERACOES_CLOSE	2	Iterações do fechamento morfológico
ALTURA_MIN/MAX_FATOR	0.25 / 0.60	Intervalo de altura dos chars no ROI
LARGURA_MIN/MAX_FATOR	0.02 / 0.25	Intervalo de largura dos chars no ROI
TESSERACT_CONFIG	PSM 7 + whitelist	Modo OCR e conjunto de chars aceitos